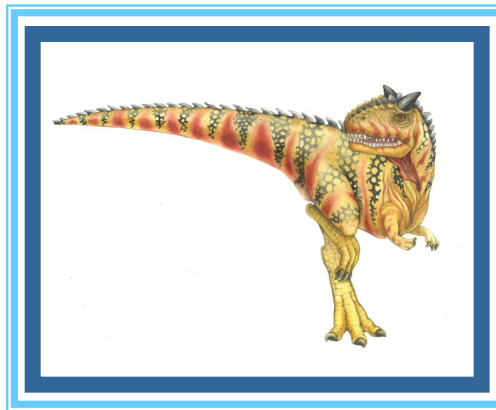


Chapter 3: Inter Process Communication (IPC)





A Unix Shell!

- Loop
 - Read command line from stdin
 - Expand wildcards
 - Interpret redirections < > |
 - pipe (as necessary), fork, dup, exec, wait





Create Your Own Unix Shell

- Command is entered and if length is non-null, keep it in history.
- Parsing : Parsing is the breaking up of commands into individual words and strings
- Checking for special characters like pipes, etc is done
- Checking if built-in commands are asked for.
- If pipes are present, handling pipes.
- Executing system commands and libraries by forking a child and calling execvp.
- Printing current directory name and asking for next input.





Question 6 in Project 1

- Main component:
 - A loop
 - Parsing the user input
 - `fork()`, `execvp()`
- Code structure (pseudocode):

```
while (command != "exit") {  
    cout<<"MiniShell>";  
    cin>>command;  
    Parsing command;  
    pid = fork();  
    if (pid < 0) erro;  
    else if (pid == 0) execvp();  
    else wait();  
}
```





Tips for Question 6 of Project 1

- You need a loop to make a shell always waiting to user input:

- ```
while(strcmp(command,"exit")!=0){
```
- ```
    cout<<"MiniShell>";
```
- ```
 cin.getline(command,50);
```
- ```
}
```

- ***A typical bug:***

- *Sometimes, if the results do not pop up immediately, but do show up when user enter the next command, then you can try to use ***fflush(stdout)****





Test “fflush”

```
■ #include <stdlib.h>
■ #include <stdio.h>
■ #include <fcntl.h>
■ int main(int argc, char **argv)
■ {
■     int pid, status;
■     int newfd;    /* new file descriptor */
■     if (argc != 2) {
■         fprintf(stderr, "usage: %s output_file\n", argv[0]);
■         exit(1);
■     }
■     if ((newfd = open(argv[1], O_CREATIO_TRUNCIO_WRONLY, 0644)) < 0) {
■         perror(argv[1]);          /* open failed */
■         exit(1);
■     }
■     printf("This goes to the standard output.\n");
■     printf("Now the standard output will go to \"%s\".\n", argv[1]);
■     fflush(stdout);
■     dup2(newfd, 1);
■     printf("This goes to the standard output too.\n");
■     exit(0);
■ }
```





Tips for Question 6 of Project 1

■ How to use execvp:

■ Syntax:

- `int execvp (const char *file, char *const argv[]);`
- `file`: points to the file name associated with the file being executed.
- `argv`: is a null terminated array of character pointers.

■ Example

- `char *cmd = "ls";`
- `char *argv[3];`
- `argv[0] = "ls";`
- `argv[1] = "-la";`
- `argv[2] = NULL;`
- `execvp(cmd, argv);`
- `//This will run "ls -la" as if it were a command`





Tips for Question 7&8 of Project 1

■ Parsing command:

- `p=strtok(command, " ");`
- `while(p&&strcmp(p,"")!=0){`
- `command_arrayA[countA]=p;`
- `countA++;`
- `p=strtok(NULL, " ");`
- `}`





Question 8 in Project 1

- How does “ls | wc” work?
- Two steps involved:
 - “ls” is executed, and some intermediate results will be obtained
 - The above intermediate results will be passed by a pipe “|” to become the new input to be processed by “wc”
- How to implement “|” in your own shell?
 - dup2(fd1, fd2) function is needed





Redirecting Standard I/O

A process that communicates solely with another process doesn't use its standard I/O.

- If process communicates with another process only via pipes, redirect standard I/O to the pipe ends
- Functions: ***dup2***

How to use?

- `#include <unistd.h>`
- `int dup2(int FD1,int FD2);`





Dup2

- For convenience...

```
dup2( int fd1, int fd2 );
```

use fd2 (new) to duplicate fd1 (old)
closes fd2 if it was in use

- Example: redirect stdin to “foo”

```
fd = open(“foo”, O_RDONLY, 0);  
dup2(fd, 0);  
close(fd);
```





Dup2 Redirect STDOUT to a File

```
■ #include <stdio.h>
■ #include <unistd.h>
■ #include <fcntl.h>

■ int main() {
■     int file = open("myfile.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
■     if (file < 0) {
■         perror("open");
■         return 1;
■     }
■     // redirecting standard output to the file
■     if (dup2(file, STDOUT_FILENO) < 0) {
■         perror("dup2");
■         close(file);
■         return 1;
■     }
■     // will be redirected to the file
■     printf("This will print in myfile.txt\n");
■     close(file);
■     return 0;
■ }
```





Redirect STDOUT to File by Executing Command

```
■ #include <stdio.h>
■ #include <stdlib.h>
■ #include <unistd.h>
■ #include <fcntl.h>

■ int main(int argc, char **argv)
■ {
■     int pid, status;
■     int newfd; /* new file descriptor */
■     char *cmd[] = { "/bin/ls", "-al", "/", 0 };
■     if (argc != 2) {
■         fprintf(stderr, "usage: %s output_file\n", argv[0]);
■         exit(1);
■     }
■     if ((newfd = open(argv[1], O_CREATIO_TRUNCIO_WRONLY, 0644)) < 0) {
■         perror(argv[1]); /* open failed */
■         exit(1);
■     }
■     printf("writing output of the command %s to \"%s\"\n", cmd[0], argv[1]);
■     dup2(newfd, 1);

■     /* now we run the command. It runs in this process and will have */
■     /* this process' standard input, output, and error */
■     execvp(cmd[0], cmd);
■     perror(cmd[0]); /* execvp failed */
■     exit(1);
■ }
```

**Try it to understand how
dup2() redirect standard
output to a file**





An Example of “Is | less”

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
int main(int argc, char* argv[])
{
    int pipefds[2];
    pid_t pid;
    if(pipe(pipefds) == -1){
        perror("pipe");
        exit(EXIT_FAILURE);
    }
    pid = fork();
    if(pid == -1){
        perror("fork");
        exit(EXIT_FAILURE);
    }
}
```

```
if(pid == 0){
    //replace stdout with the write end of the pipe
    dup2(pipefds[1],STDOUT_FILENO);
    //close read to pipe, in child
    close(pipefds[0]);
    execlp("ls","ls",NULL);
    exit(EXIT_SUCCESS);
}else{
    //Replace stdin with the read end of the pipe
    dup2(pipefds[0],STDIN_FILENO);
    //close write to pipe, in parent
    close(pipefds[1]);
    execlp("less","less",NULL);
    exit(EXIT_SUCCESS);
}
}
```





Tips for Question 8 of Project 1

■ How to digest “|” in your shell:

- fork A
if(pid A<0)
{...}
else if(pid A==0)
{ **1. Redirect STDOUT to the write end of the pipe**
- **2. Execute command before “|” }**
else
{ fork B
if (pid B<0)
{...}
else if (pid B==0)
{**1. Redirect the read end of the pipe to STDIN**
- **2. Execute the arg after “|” }**
else
{...}
}

